



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS,
ELECTRÓNICA E INDUSTRIAL



INFORME DEL PROYECTO

GRUPO 04

ASIGNATURA

Estructura de Datos

DOCENTE

Ing. José Caiza, Mg.

TEMA

SMARTETL-DS + IA

Sistema ETL Inteligente con Estructuras de Datos,
Árboles, Grafos e Inteligencia Artificial

ALUMNOS

Camacho Monta Josue Jampier
Chalco Tasna Kenneth Mateo
Cunalata Mendoza Damian Alexander
Silva Camuendo Luis Alexander
Tacuri Santillan Monica Sara
Tisalema Guashco Darwin Joel

CURSO

Tercero - B

Ambato - Ecuador
2026



INFORME DEL PROYECTO

1. Introducción

El manejo de grandes volúmenes de información requiere estructuras y algoritmos capaces de almacenar, recorrer y consultar los datos de manera organizada. En la asignatura Estructura de Datos, el proyecto SmartETL-DS + IA permite aplicar estos conocimientos mediante el desarrollo de un sistema en Java que procesa información real proveniente de archivos de datos.

El proyecto utiliza registros relacionados con casos de COVID-19 de Estados Unidos durante el año 2021. El archivo principal contiene más de 1,18 millones de registros, complementados con información de condados y relaciones de vecindad entre ellos. Estos datos presentan diferentes situaciones de calidad, como códigos FIPS faltantes, valores vacíos y registros especiales, lo que permite trabajar con un escenario cercano al procesamiento de información real. [1]

SmartETL-DS + IA se basa en el proceso ETL (Extract, Transform y Load). En esta primera entrega se desarrolla principalmente la fase Extract, encargada de verificar, leer y convertir la información de los archivos en objetos Java para almacenarlos utilizando estructuras de datos implementadas por el propio equipo. En posteriores etapas se incorporarán procesos de transformación, carga, árboles, grafos e Inteligencia Artificial.

2. Descripción general

SmartETL-DS + IA es un proyecto integrador desarrollado en Java cuyo propósito es aplicar estructuras de datos propias al procesamiento de información proveniente de archivos CSV y TSV. **El sistema está diseñado siguiendo el modelo ETL:** extracción, transformación y carga de datos.

En su primera etapa, el sistema verifica la existencia y estructura de los archivos, procesa cada fila, crea objetos Java y los almacena principalmente en una lista enlazada desarrollada por el equipo. Además, permite realizar búsquedas, consultar estadísticas, identificar posibles incidencias en los datos y comparar el comportamiento de diferentes estructuras.

En las siguientes entregas el proyecto incorporará nuevas estructuras y funcionalidades, como cola, pila, árbol binario de búsqueda, grafos, recorridos BFS y DFS y una capa de Inteligencia Artificial para la generación de reportes.

3. Planteamiento del problema

Los conjuntos de datos reales pueden contener grandes cantidades de registros y presentar valores incompletos, inconsistencias o información que requiere validación antes de ser utilizada.

En SmartETL-DS + IA se trabaja con más de un millón de registros diarios de COVID-19, además de información de condados y relaciones de vecindad. Procesar esta cantidad de información permite evidenciar las diferencias entre las estructuras de datos utilizadas y analizar el costo de operaciones como inserción, recorrido y búsqueda.

El problema consiste en desarrollar una solución que permita leer, validar, almacenar y consultar estos registros utilizando estructuras implementadas por el propio equipo, sin delegar estas funciones a una base de datos o a colecciones prediseñadas de Java.



4. Objetivos

General:

- Diseñar e implementar la primera etapa del sistema SmartETL-DS + IA en Java, aplicando estructuras de datos desarrolladas por el equipo para extraer, validar, almacenar y consultar información proveniente de archivos reales de casos de COVID-19.

Específicos:

- Implementar estructuras de datos propias en Java para almacenar y gestionar los registros procesados por el sistema.
- Desarrollar la fase Extract, validando los archivos de entrada, convirtiendo sus registros en objetos y detectando posibles incidencias en los datos.
- Implementar búsquedas, consultas y comparaciones de estructuras mediante el menú principal, documentando los resultados obtenidos en esta primera entrega.

5. Alcance de la primera entrega

La primera entrega de SmartETL-DS + IA corresponde principalmente a la fase **Extract** del proceso ETL.

Durante esta etapa se implementan:

- verificación de los archivos de entrada;
- validación de las cabeceras;
- lectura de archivos CSV y TSV;
- creación de objetos Java a partir de cada registro;
- almacenamiento mediante ListaEnlazada;
- detección y conteo de incidencias;
- búsquedas secuenciales;
- estadísticas básicas;
- comparación entre ListaEnlazada y ListaSecuencial;
- menú de interacción con el usuario.

En esta entrega **no se implementan todavía** la cola, pila, fase Transform, fase Load, ordenamiento, árbol BST, grafo, recorridos BFS y DFS ni el módulo de Inteligencia Artificial. Estos elementos forman parte de la arquitectura general del proyecto y serán desarrollados en los siguientes hitos.

6. Resultados obtenidos

Integrantes y roles del equipo

Integrante	Rol	Responsabilidad principal	Rama
Cunalata Mendoza Damian Alexander	Líder técnico e integración	README, preparar_base.py, datos, Busqueda, scripts e integración	Damian_Cunalata
Chalco Tasna Kenneth Mateo	Desarrollo - estructuras	Nodo, ListaEnlazada y ListaSecuencial	Kenneth_Chalco / Mateo-Chalco



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



Tisalema Guashco Darwin Joel	Desarrollo - Extract	Lectura CSV/TSV, validación y conteo de incidencias	Rama-Joel
Silva Camuendo Luis Alexander	Desarrollo - modelo	Registro, Esquema, CasoDiario, Condado y Vecindad	Rama-Luis-Silva
Tacuri Santillan Mónica Sara	Desarrollo - programa principal y pruebas	Main y PruebaListaEnlazada	Sara-Tacuri
Camacho Monta Josue Jampier	Documentación	Documento, diagrama de clases, arquitectura y capturas	rama---Josue

6.1. Ejercicio

Construir la primera entrega (fase Extract) del proyecto SmartETL-DS + IA: un sistema que lee más de 1.185.373 registros diarios de casos de COVID-19 por condado de Estados Unidos (además de un catálogo de condados y una tabla de vecindad entre condados), y los almacena en estructuras de datos programadas por el propio equipo, sin usar ArrayList, LinkedList, Stack, Queue ni HashMap de java.util. El sistema debe verificar que existan los archivos de entrada, validar que sus columnas coincidan con el esquema esperado, convertir cada fila en un objeto Java, cargar esos objetos en una lista enlazada propia, contar los posibles problemas que trae la base, y permitir consultar los registros mediante un menú con búsquedas y estadísticas.

6.2. Solución propuesta

El código de esta entrega está organizado en cuatro paquetes más la clase Main, sumando doce clases en total: model (el molde de cada registro), estructuras (la lista enlazada propia y la lista secuencial de comparación), etl (la lectura de los archivos) y algoritmos (la búsqueda con conteo de comparaciones).

Paquete / Clase	Responsabilidad
model.Registro (interfaz)	Contrato común de todo registro: getClave() y mostrar(). Permite tratar CasoDiario, Condado y Vecindad de forma polimórfica.
model.Eschema	Guarda la cabecera exacta esperada de cada archivo (casos_diarios.csv, condados.csv, vecindad.tsv) y valida que la cabecera leída coincida.
model.CasoDiario	Una fila de casos_diarios.csv: fecha, condado, estado, fips, casos, muertes. Implementa Registro.
model.Condado	Una fila de condados.csv: fips, nombre, estado. Implementa Registro y Comparable<Condado> (por fips).
model.Vecindad	Una fila de vecindad.tsv: condado de origen y de destino. Implementa Registro.
estructuras.Nodo<T>	Nodo genérico con un dato y una referencia al siguiente nodo.
estructuras.ListaEnlazada<T>	Estructura principal de esta entrega: lista enlazada simple con cabeza, cola y tamaño; implementa Iterable<T>.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



estructuras.ListaSecuencial<T>	Lista basada en arreglo (Object[]) que duplica su capacidad al llenarse; se usa solo para comparar el rendimiento contra la lista enlazada.
etl.Extract	Lee casos_diarios.csv, condados.csv y vecindad.tsv, valida la cabecera de cada uno, arma los objetos y cuenta los distintos tipos de incidencias de calidad de datos.
algoritmos.Busqueda	Búsqueda secuencial sobre ListaEnlazada<CasoDiario>: por FIPS + fecha (se detiene en la primera coincidencia) y por nombre de condado (devuelve todas las coincidencias); cuenta las comparaciones realizadas.
test.PruebaListaEnlazada	Clase de prueba que verifica el funcionamiento de ListaEnlazada mediante casos normales y casos límite, como lista vacía, inserción, obtención y eliminación de elementos; muestra resultados OK o FALLA según corresponda.
Main	Verifica que existan los tres archivos, muestra el menú de 10 opciones (cargar, mostrar, buscar, incidencias, estadísticas, condados/vecindad, comparar listas, ver esquema, salir) y coordina Extract y Busqueda.

Diferencia frente a la planificación inicial: el enunciado agrupaba las clases en model, estructuras, etl, algoritmos y Main, lo cual coincide exactamente con el código real revisado; no se encontró ningún paquete adicional (todavía no existen Transform, Load, Cola, Pila, Árbol ni Grafo en esta entrega, tal como se advirtió en el contexto del proyecto).

6.2.1. Fuente y origen de los datos

Archivo	Origen	Filas	Enlaces
casos_diarios.csv	The New York Times - covid-19-data	1.185.373	Repositorio fuente / Descarga directa
condados.csv	kjhealy/fips-codes	3.195	Repositorio fuente / Descarga directa
vecindad.tsv	turibe/us-county-adjacency	22.200	Repositorio fuente / Descarga directa

6.3. ¿Qué son las estructuras utilizadas y por qué se aplican aquí?

- **Lista enlazada (ListaEnlazada<T>):** cada elemento se guarda en un Nodo<T> con una referencia al siguiente nodo; la lista solo conserva la cabeza y la cola. Insertar al final es O(1) porque no hace falta recorrer nada (se usa la referencia a la cola); no reserva memoria de más ni necesita copiarse al crecer, lo que la hace apropiada para cargar más de un millón de registros sin desperdiciar memoria. Es la estructura obligatoria de esta entrega y la que usan Extract, Busqueda y Main.
- **Lista secuencial (ListaSecuencial<T>):** guarda los datos en un arreglo interno (Object[]) que empieza en 16 posiciones y duplica su tamaño (crecer()) cuando se llena, copiando todo el contenido anterior. Se usa exclusivamente para comparar su comportamiento contra la lista enlazada (opción 8 del menú), no para el flujo principal del programa.



6.4. Relación entre requisitos y código

Requisito	Aplicación en el código
Flujo CSV → Extract → Objetos Java → ListaEnlazada → Menú	Extract.cargarCasos()/cargarCondados()/cargarVecindad() devuelven ListaEnlazada<Registro>; Main las consume desde el menú.
No usar colecciones de java.util	Ninguna clase importa ArrayList, LinkedList, Stack, Queue ni HashMap; Nodo/ListaEnlazada/ListaSecuencial están escritas desde cero.
Verificar archivos antes de leer	Main.verificarArchivos() comprueba la existencia de los 3 archivos antes de permitir cualquier opción del menú.
Validar columnas esperadas	Esquema.cabeceraValida() compara la cabecera leída contra CABECERA_CASOS_DIARIOS / CABECERA_CONDAOS / CABECERA_VECINDAD.
Detectar y contar problemas de los datos (sin corregirlos aún)	Extract cuenta casosSinFips, casosCondaoUnknown, casosMuertesVacias, condadosNoSonCondao, condadosFipsSinCero, vecindadVecinoDeSiMismo, vecindadRepetida, y registra cada uno como texto en la lista incidencias.
Consultar registros mediante un menú	Main.mostrarMenu() con 10 opciones: cargar, mostrar, buscar por FIPS+fecha, buscar por nombre, ver incidencias, estadísticas, condados/vecindad, comparar listas, ver esquema, salir.
Búsquedas con conteo de comparaciones	Busqueda.porFipsYFecha() y Busqueda.porCondao() incrementan comparaciones en cada iteración; Main las imprime tras cada búsqueda.
CasoDiario, Condao y Vecindad implementan Registro	Confirmado en el código real: las tres clases declaran implements Registro y sobrescriben getClave() y mostrar().
ListaEnlazada utiliza Nodo	Confirmado: ListaEnlazada<T> guarda cabeza/cola como Nodo<T> y los enlaza en insertar()/eliminar().
Main utiliza Extract y Busqueda	Confirmado: Main mantiene una instancia de Extract y crea una Busqueda por cada consulta.

6.5. Paso 1: Definir el contrato común de los registros (Registro y Esquema)

```
public interface Registro {  
    String getClave();  
    String mostrar();  
}  
  
public final class Esquema {  
    public static final String[] CABECERA_CASOS_DIARIOS =  
        {"date", "county", "state", "fips", "cases", "deaths"};  
    public static boolean cabeceraValida(String[] leida, String[] esperada) { /* ... */ }  
}
```

Registro es lo que permite que Extract y Main traten a CasoDiario, Condao y Vecindad de la misma forma (polimorfismo) sin conocer su tipo concreto. Esquema centraliza las cabeceras exactas de los tres archivos, de modo que un archivo con columnas distintas se rechaza en vez de leerse mal.



6.6. Paso 2: Implementar las clases de datos (CasoDiario, Condado, Vecindad)

```
public class CasoDiario implements Registro {
    private final String fecha, condado, estado, fips;
    private final int casos, muertes;

    public static CasoDiario desdeTexto(String fecha, String condado, String estado,
        String fips, String casosTexto, String muertesTexto) {
        int casos = (casosTexto == null || casosTexto.isBlank()) ? -1 :
Integer.parseInt(casosTexto.trim());
        // muertes vacio se interpreta como -1 (dato faltante)
        ...
    }

    @Override public String getClave() { return fips + "-" + fecha; }
}
```

Cada clase de datos guarda sus atributos como final (inmutables una vez creado el objeto), implementa equals()/hashCode() y sobrescribe getClave() con la llave natural de su archivo: fips+fecha para CasoDiario, fips para Condado, y fipsOrigen+fipsDestino para Vecindad. Condado además implementa Comparable<Condado> comparando por fips, dejado listo para una futura estructura ordenada (árbol BST) sin que eso exista todavía en esta entrega.

6.7. Paso 3: Implementar Nodo y ListaEnlazada

```
public class Nodo<T> {
    private T dato;
    private Nodo<T> siguiente;
}

public class ListaEnlazada<T> implements Iterable<T> {
    private Nodo<T> cabeza, cola;
    private int tamaño;

    public void insertar(T dato) {
        Nodo<T> nuevo = new Nodo<>(dato);
        if (cabeza == null) { cabeza = nuevo; cola = nuevo; }
        else { cola.setSiguiente(nuevo); cola = nuevo; }
        tamaño++;
    }
}
```

Guardar una referencia a la cola es la decisión clave: sin ella, insertar() tendría que recorrer toda la lista cada vez ($O(n)$) para llegar al final; con ella, insertar más de un millón de registros cuesta $O(1)$ por registro. ListaEnlazada implementa Iterable<T> para poder recorrerse con for-each en Extract, Busqueda y Main.

6.8. Paso 4: Implementar ListaSecuencial para comparar

```
public class ListaSecuencial<T> implements Iterable<T> {
    private Object[] datos;
    private int tamaño;
    private static final int CAPACIDAD_INICIAL = 16;
```



```
public void insertar(T dato) {  
    if (tamano == datos.length) crecer(); // duplica el arreglo y copia todo  
    datos[tamano++] = dato;  
}  
}
```

ListaSecuencial no participa en la carga real de datos: existe únicamente para que Main.compararListas() inserte los mismos registros en ambas estructuras y mida el tiempo de cada una, evidenciando en la práctica la diferencia entre un arreglo que se duplica y una lista enlazada que solo referencia nodos.

6.9. Paso 5: Implementar Extract (lectura y validación de los tres archivos)

```
public ListaEnlazada<CasoDiario> cargarCasos(String ruta) throws IOException {  
    ListaEnlazada<CasoDiario> lista = new ListaEnlazada<>();  
    // 1) leer la cabecera y compararla con Esquema.CABECERA_CASOS_DIARIOS  
    // 2) por cada linea: separar por comas (respetando comillas), limpiar espacios  
    // 3) contar incidencias (fips vacio, condado Unknown, muertes vacias) sin corregirlas  
    // 4) lista.insertar(new CasoDiario(...))  
    return lista;  
}
```

Extract implementa su propio separador de columnas (separar()), que respeta comillas para no romper campos con comas internas, y usado tanto para CSV (coma) como para TSV (tabulación). Cada archivo tiene su propio contador de filas leídas/cargadas/descartadas y sus propios contadores de incidencias específicas, todos disponibles mediante getters para que Main los muestre en el resumen de carga.

6.10. Paso 6: Implementar Búsqueda con conteo de comparaciones

```
public CasoDiario porFipsYFecha(ListaEnlazada<CasoDiario> lista, String fips, String fecha) {  
    comparaciones = 0;  
    for (CasoDiario caso : lista) { // for-each: nodo por nodo, nunca obtener(i) en un for  
        comparaciones++;  
        if (fips.equals(caso.getFips()) && fecha.equals(caso.getFecha())) return caso;  
    }  
    return null;  
}
```

El propio comentario del código explica una decisión de diseño importante: recorrer con el iterador for-each cuesta $O(n)$, pero llamar a lista.obtener(i) dentro de un for volvería a caminar desde la cabeza en cada vuelta, degradando la búsqueda a $O(n^2)$. porFipsYFecha() se detiene en la primera coincidencia (la pareja fips+fecha es única); porCondado() no se detiene, porque un condado tiene un registro por cada día del año.

6.11. Paso 7: Implementar Main (verificación de archivos y menú)

```
public static void main(String[] args) {  
    if (!verificarArchivos()) return;  
    while (opcion != 0) {
```



```
mostrarMenu();  
opcion = leerOpcion(teclado);  
switch (opcion) { case 1: cargarCasos(); break; /* ...2 a 9... */ }  
}  
}
```

Main mantiene el estado de la sesión (casos, condados, vecindad ya cargados) para no releer los archivos dos veces, y delega toda la lógica de lectura y búsqueda en Extract y Busqueda respectivamente, limitándose a coordinar el menú y presentar los resultados en consola.

6.12. Tipos de datos utilizados

Variables	Tipo / Categoría	Descripción / Uso
casos, muertes, fechasDistintas, tamaño, comparaciones y los contadores de Extract	int (primitivo)	Estadísticas y contadores numéricos enteros.
esAutoVecindad(), estaVacía(), cabeceraValida()	boolean (primitivo)	Resultados lógicos de validación.
fecha, condado, estado, fips, nombre (Condado/Vecindad)	String (referencia)	Texto proveniente directamente de las columnas de los archivos.
dato, T (Nodo, ListaEnlazada, ListaSecuencial)	Genérico <T> (referencia)	Permite que la misma lista/nodo almacene CasoDiario, Condado, Vecindad o String según el caso.
cabeza, cola, siguiente (ListaEnlazada/Nodo)	Referencias a nodo	Sostienen el encadenado de la lista enlazada.
datos: Object[] (ListaSecuencial)	Arreglo de objetos (referencia)	Almacenamiento interno de la lista secuencial de comparación.

6.13. Diagrama de clases

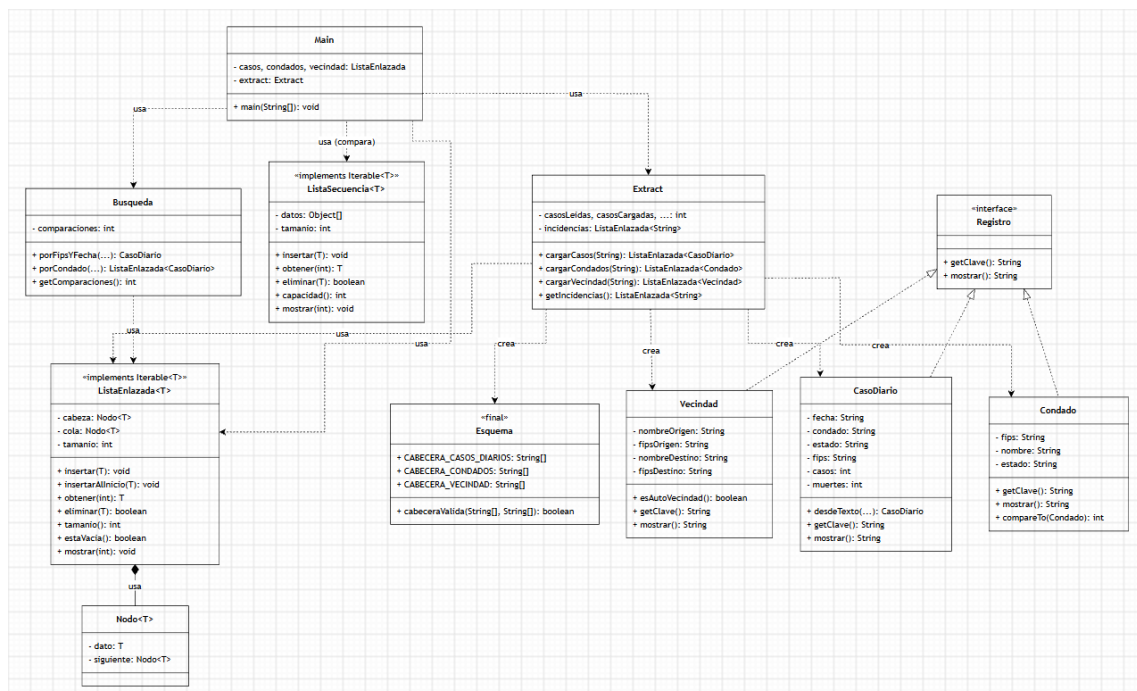


Figura 1. Diagrama de clases de la primera entrega de SmartETL-DS (12 clases reales, agrupadas por paquete).

Las 12 clases están agrupadas visualmente en sus cinco paquetes reales: **modelo** (Registro, Esquema, CasoDiario, Condado, Vecindad), **estructuras** (Nodo, ListaEnlazada, ListaSecuencial), **etl** (Extract), **algoritmos** (Busqueda) y **Main** como raíz del programa. **El diagrama respeta exactamente lo confirmado en el código:** CasoDiario, Condado y Vecindad implementan Registro (flecha punteada con triángulo hueco); ListaEnlazada usa Nodo (agregación); Extract usa Esquema y crea CasoDiario/Condado/Vecindad, además de usar ListaEnlazada; Main usa Extract, Busqueda, ListaEnlazada y ListaSecuencial.

6.14. Especificación de las estructuras principales

Estructura	Estado	Operaciones principales
ListaEnlazada<T>	cabeza: Nodo<T>, cola: Nodo<T>, tamanio: int	insertar, insertarAlInicio, obtener, eliminar, tamanio, estaVacia, mostrar, iterator
ListaSecuencial<T>	datos: Object[], tamanio: int, CAPACIDAD_INICIAL = 16	insertar, obtener, eliminar, tamanio, estaVacia, capacidad, mostrar, iterator

Invariantes:

- cabeza es null si y solo si la lista está vacía ($\text{tamanio} == 0$); lo mismo aplica a cola.
- cola.getSiguiente() siempre es null (la cola nunca apunta a otro nodo).
- Tras insertar(), el nodo anterior a cola siempre queda enlazado correctamente y cola se actualiza al nuevo nodo.
- Tras eliminar() el elemento que era cola, la referencia cola se actualiza al nodo anterior (o a null si la lista queda vacía).
- obtener(posicion) exige $0 \leq \text{posicion} < \text{tamanio}$; fuera de ese rango lanza `IndexOutOfBoundsException`.



6. En ListaSecuencial, las posiciones válidas son siempre datos[0 .. tamaño-1]; el resto puede estar sin usar.
7. En ListaSecuencial, tamaño nunca excede datos.length; al llegar al límite, crecer() duplica la capacidad antes de insertar.
8. El iterador de ambas listas nunca se apoya en obtener(i) dentro de un bucle externo, para no degradar un recorrido de $O(n)$ a $O(n^2)$.

6.15. Diagrama de flujo del programa principal (Main)

INICIO

SI NO verificarArchivos() ENTONCES mostrar archivo faltante y TERMINAR

REPETIR

mostrar menu de 10 opciones

leer opcion

SEGUN opcion HACER

1: cargarCasos() (Extract.cargarCasos -> ListaEnlazada<CasoDiario>)

2: mostrarRegistros() (primeros y ultimos 5 registros)

3: buscarPorFipsYFecha() (Busqueda.porFipsYFecha + comparaciones)

4: buscarPorNombre() (Busqueda.porCondado + comparaciones)

5: mostrarIncidencias()

6: mostrarEstadisticas() (fechas distintas, memoria usada)

7: verCondadosYVecindad() (Extract.cargarCondados / cargarVecindad)

8: compararListas() (ListaEnlazada vs ListaSecuencial)

9: mostrarEsquema()

0: salir

HASTA QUE opcion = 0

FIN

6.16. Prueba de escritorio

Caso de prueba tomado del archivo real test/PruebaListaEnlazada.java (9 pruebas automatizadas sobre ListaEnlazada<String>):

Prueba	Operaciones	Resultado esperado
Lista vacía	new ListaEnlazada<>()	estaVacía() = true, tamaño() = 0.
Insertar uno	insertar("Ana")	tamaño() = 1, obtener(0) = "Ana".
Insertar varios	insertar("Ana"), insertar("Luis"), insertar("Zoe")	tamaño() = 3, orden respetado: Ana, Luis, Zoe.
Obtener por posición	insertar(10,20,30) sobre ListaEnlazada<Integer>	obtener(1) = 20.
Eliminar el primero	eliminar("Ana") sobre [Ana, Luis, Zoe]	tamaño() = 2; obtener(0) = "Luis".
Eliminar del medio	eliminar("Luis") sobre [Ana, Luis, Zoe]	tamaño() = 2; quedan Ana y Zoe en ese orden.
Eliminar el último	eliminar("Zoe") y luego insertar("Nuevo")	tamaño() = 3; obtener(2) = "Nuevo" (la cola quedó bien reenlazada).
Eliminar inexistente	eliminar("Carlos") sobre [Ana, Luis]	Retorna false; tamaño() sigue en 2 (no lanza error).
Tamaño final	insertar A,B,C; eliminar(B); insertar D	tamaño() = 3.

Las nueve pruebas están automatizadas y se ejecutan con PruebaListaEnlazada.main(), que imprime OK/FALLA por cada una y un resumen final de "Pruebas OK" / "Pruebas con FALLA".



Adicionalmente, la carga real de casos_diarios.csv (documentada en el README del proyecto, con valores ya verificados por el equipo) deja evidencia de las incidencias que Extract detecta y cuenta sin corregir todavía:

Archivo	Problema detectado	Cantidad
casos_diarios.csv	deaths vacío	28.470
casos_diarios.csv	Sin código FIPS	10.803
casos_diarios.csv	county = Unknown	9.708
condados.csv	Fila de país mezclada con los condados	1
condados.csv	Filas de estados mezcladas con los condados	51
condados.csv	FIPS de 4 dígitos, sin el cero inicial	316
vecindad.tsv	Condado vecino de sí mismo	3.234
vecindad.tsv	Vecindad repetida en los dos sentidos	9.483

Nota: estos números provienen del README del repositorio (aportados por Damian y Joel a partir de la ejecución real del programa) y no fueron inventados.

6.17. Diagrama de arquitectura propuesta

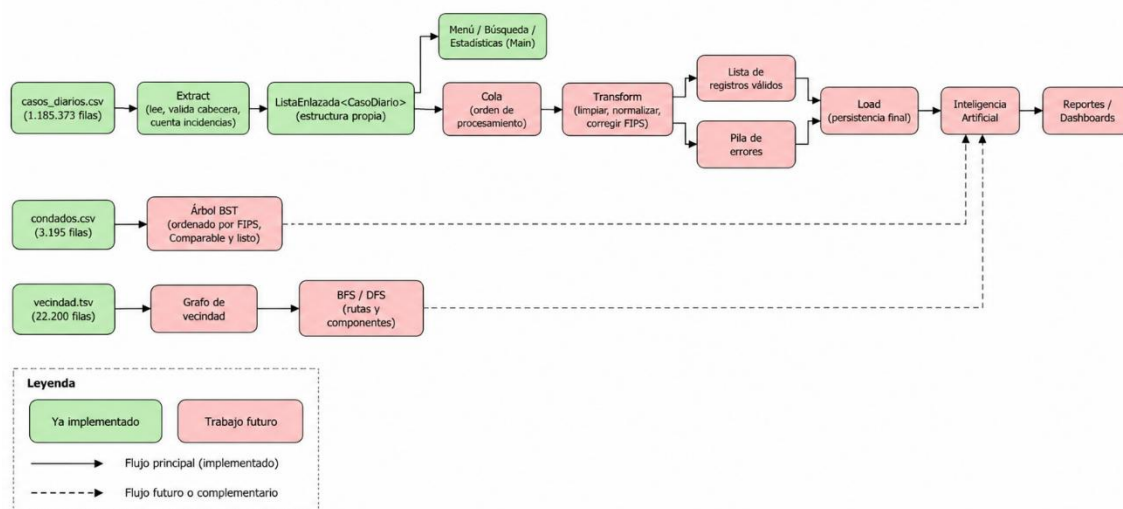


Figura 2. Arquitectura propuesta de SmartETL-DS + IA (verde = implementado, rojo = trabajo futuro).

Implementado actualmente (verde):

- casos_diarios.csv, condados.csv y vecindad.tsv como archivos de entrada.
- Extract, que lee y valida los tres archivos y cuenta sus incidencias.
- ListaEnlazada<CasoDiario>, la estructura donde quedan cargados los registros.
- El menú de consola de Main (opciones 1 a 9): mostrar, buscar, ver incidencias, estadísticas, comparar listas, ver esquema.

Trabajo futuro (rojo, sin programar todavía):

- **Cola:** ordenará el procesamiento de los registros extraídos antes del Transform.
- **Transform:** limpiará y normalizará los datos (por ejemplo, completar el FIPS de 4 dígitos con un cero) y separará los registros válidos de los erróneos.
- **Lista de registros válidos y Pila de errores:** destino de la fase Transform, según corresponda a cada registro.
- **Load:** persistencia final de los datos ya transformados.



- **Árbol BST sobre condados.csv:** ya está preparado el terreno, porque Condado implementa Comparable<Condado> comparando por fips.
- Grafo sobre vecindad.tsv, recorrido con BFS y DFS, para explotar las relaciones de vecindad entre condados.
- Módulo de Inteligencia Artificial, que tomará los resultados de Load, del árbol y del grafo para generar reportes.

Nota: esta arquitectura es la propuesta de solución completa que exige el enunciado para el 1 de octubre; en el código de esta primera entrega solo existen los elementos marcados en verde. Ninguno de los módulos en rojo (Cola, Pila, Transform, Load, Árbol BST, Grafo, BFS, DFS, IA) está implementado todavía.

7. Conclusiones

- La primera entrega de SmartETL-DS implementa correctamente la fase Extract, permitiendo validar los archivos de entrada, convertir los registros en objetos Java y almacenarlos en estructuras de datos propias.
- ListaEnlazada funciona como la estructura principal del sistema y permite insertar registros de forma eficiente al mantener una referencia directa a la cola, evitando redimensionamientos y copias propias de una estructura basada en arreglo.
- La separación del proyecto en clases con responsabilidades específicas, como model, Extract, Búsqueda y Main, facilita la organización, mantenimiento e integración del sistema.

8. Recomendaciones

- Verificar nuevamente los resultados utilizando la base de datos completa antes de la entrega final, para asegurar que los conteos y estadísticas coincidan con la ejecución real.
- Integrar en la rama main todo el código definitivo del equipo y comprobar que el proyecto compile y funcione correctamente desde una sola versión.
- Mantener actualizados los diagramas, capturas y documentación cada vez que se realice un cambio en el código, para que la documentación siempre represente el sistema real.

9. Referencias bibliográficas

- [1] The New York Times, "Coronavirus (Covid-19) Data in the United States," GitHub repository, 2021. Accedido el 27 de septiembre de 2026. [En línea]. Disponible: <https://github.com/nytimes/covid-19-data>
- [2] Baeldung, "Reduce Object Header Size and Save Memory in Java 25," Baeldung, may 2026. Accedido el 27 de septiembre de 2026. [En línea]. Disponible: <https://www.baeldung.com/java-object-header-reduced-size-save-memory>
- [3] "The Cost of Garbage Collection for State Machine Replication," arXiv preprint, arXiv:2405.11182, 2024. Accedido el 27 de septiembre de 2026. [En línea]. Disponible: <https://arxiv.org/pdf/2405.11182>

10. Anexos

En el siguiente enlace se encuentra disponible el repositorio del proyecto SmartETL-DS + IA, el cual contiene el código fuente en Java, las capturas de ejecución y los diagramas elaborados como evidencia del desarrollo.

Enlace del repositorio: https://github.com/Damienq7w/Estructura_Datos_SmartETL